CSC2044 Concurrent Programming in Java

Sample Solution Document

Concurrent Concert Ticket Booking System

Course Code	CSC2044
Course Title	Concurrent Programming in Java
Sample Problem	Concurrent Concert Ticket Booking System
Main Concepts	Sequential processing, concurrent processing, shared data, race condition, synchronization
Programming Language	Java
Purpose	To show a proper sample solution using a different problem from the uploaded assignment

Important note: This is a sample solution for demonstration only. It does not solve the uploaded bank account transaction problem. Instead, it uses a different ticket booking scenario while demonstrating the same key concurrency concepts.

1. Problem Scenario

A concert ticketing platform receives different ticket-related requests at the same time. Customers may book tickets, cancel tickets, check availability, or the organiser may reserve some tickets for official guests. If these requests are processed one by one, the result is predictable. However, if several requests are processed by different threads at the same time, the shared ticket inventory may be updated incorrectly unless synchronization is used.

This sample solution demonstrates sequential processing, concurrent processing using Java threads, shared heap data, a race condition, and a synchronized solution.

2. System Design

Component	Description
TicketInventory	Stores the shared number of available tickets.
TicketOperation	Represents booking, cancellation, availability checking, or organiser ticket hold.
TicketWorker	A Runnable task that processes one ticket operation using a thread.
availableTickets	Shared heap data accessed by multiple threads.
UnsafeTicketSalesCounter	Demonstrates a race condition without synchronization.
SafeTicketSalesCounter	Solves the race condition using a synchronized method.

3. Complete Java Program

File name: ConcertTicketBookingSystem.java

```
import java.util.Arrays;
import java.util.List;

public class ConcertTicketBookingSystem {

    enum OperationType {
        BOOK, CANCEL, CHECK, SERVICE_HOLD
    }

    static class TicketOperation {
        private final int operationId;
        private final OperationType type;
        private final int quantity;
        private final int processingTimeMs;

        public TicketOperation(int operationId, OperationType type, int quantity, int processingTimeMs) {
            this.operationId = operationId;
            this.type = type;
            this.quantity = quantity;
            this.processingTimeMs = processingTimeMs;
        }

        public int getOperationId() {
            return operationId;
        }

        public OperationType getType() {
            return type;
        }

        public int getQuantity() {
            return quantity;
        }

        public int getProcessingTimeMs() {
            return processingTimeMs;
        }
```

```
    public String getDescription() {
        switch (type) {
            case BOOK:
                return "Book " + quantity + " ticket(s)";
            case CANCEL:
                return "Cancel " + quantity + " ticket(s)";
            case CHECK:
                return "Check available tickets";
            case SERVICE_HOLD:
                return "Reserve " + quantity + " ticket(s) for organiser hold";
            default:
                return "Unknown operation";
        }
    }
}

static class TicketInventory {
    private int availableTickets;

    public TicketInventory(int availableTickets) {
        this.availableTickets = availableTickets;
    }

    public int getAvailableTickets() {
        return availableTickets;
    }

    public void processSequential(TicketOperation operation) throws InterruptedException {
        System.out.println("Starting Operation " + operation.getOperationId() + ": "
                + operation.getDescription());

        Thread.sleep(operation.getProcessingTimeMs());
        applyOperation(operation);

        System.out.println("Finished Operation " + operation.getOperationId());
        System.out.println("Available tickets: " + availableTickets);
        System.out.println();
    }

    public void processConcurrentSafely(TicketOperation operation) throws InterruptedException {
        String threadName = Thread.currentThread().getName();

        System.out.println(threadName + " started Operation " + operation.getOperationId()
                + ": " + operation.getDescription());

        Thread.sleep(operation.getProcessingTimeMs());

        synchronized (this) {
            applyOperation(operation);
            System.out.println(threadName + " updated inventory. Available tickets: "
                    + availableTickets);
        }

        System.out.println(threadName + " finished Operation " + operation.getOperationId());
    }

    private void applyOperation(TicketOperation operation) {
        switch (operation.getType()) {
            case BOOK:
                if (availableTickets >= operation.getQuantity()) {
                    availableTickets = availableTickets - operation.getQuantity();
                } else {
                    System.out.println("Not enough tickets available for booking.");
                }
                break;

            case CANCEL:
                availableTickets = availableTickets + operation.getQuantity();
                break;

            case CHECK:
                System.out.println("Ticket check: " + availableTickets + " ticket(s) available.");
                break;

            case SERVICE_HOLD:
                if (availableTickets >= operation.getQuantity()) {
                    availableTickets = availableTickets - operation.getQuantity();
```

```
                } else {
                    System.out.println("Not enough tickets available for organiser hold.");
                }
                break;
            }
        }
    }

    static class TicketWorker implements Runnable {
        private final TicketInventory inventory;
        private final TicketOperation operation;

        public TicketWorker(TicketInventory inventory, TicketOperation operation) {
            this.inventory = inventory;
            this.operation = operation;
        }

        @Override
        public void run() {
            try {
                inventory.processConcurrentSafely(operation);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }

    static class UnsafeTicketSalesCounter {
        private int soldTickets = 0;

        public void sellOneTicketUnsafe() {
            int currentValue = soldTickets;
            Thread.yield();
            soldTickets = currentValue + 1;
        }

        public int getSoldTickets() {
            return soldTickets;
        }
    }

    static class SafeTicketSalesCounter {
        private int soldTickets = 0;

        public synchronized void sellOneTicketSafely() {
            int currentValue = soldTickets;
            soldTickets = currentValue + 1;
        }

        public int getSoldTickets() {
            return soldTickets;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        runSequentialExample();
        runConcurrentExample();
        runUnsafeRaceConditionExample();
        runSynchronizedRaceConditionExample();
    }

    private static List<TicketOperation> createSampleOperations() {
        return Arrays.asList(
                new TicketOperation(1, OperationType.BOOK, 10, 700),
                new TicketOperation(2, OperationType.CANCEL, 3, 500),
                new TicketOperation(3, OperationType.CHECK, 0, 300),
                new TicketOperation(4, OperationType.SERVICE_HOLD, 5, 900),
                new TicketOperation(5, OperationType.BOOK, 8, 600),
                new TicketOperation(6, OperationType.CANCEL, 2, 400),
                new TicketOperation(7, OperationType.CHECK, 0, 200),
                new TicketOperation(8, OperationType.BOOK, 12, 800)
        );
    }

    private static void runSequentialExample() throws InterruptedException {
        System.out.println("========== SEQUENTIAL TICKET PROCESSING ==========");
        TicketInventory inventory = new TicketInventory(100);
```

```
    for (TicketOperation operation : createSampleOperations()) {
        inventory.processSequential(operation);
    }

    System.out.println("Final available tickets after sequential processing: "
            + inventory.getAvailableTickets());
    System.out.println();
}

private static void runConcurrentExample() throws InterruptedException {
    System.out.println("========== CONCURRENT TICKET PROCESSING ==========");
    TicketInventory inventory = new TicketInventory(100);
    List<TicketOperation> operations = createSampleOperations();
    Thread[] workers = new Thread[operations.size()];

    for (int i = 0; i < operations.size(); i++) {
        workers[i] = new Thread(new TicketWorker(inventory, operations.get(i)),
                "TicketWorker-" + (i + 1));
        workers[i].start();
    }

    for (Thread worker : workers) {
        worker.join();
    }

    System.out.println("Final available tickets after concurrent processing: "
            + inventory.getAvailableTickets());
    System.out.println();
}

private static void runUnsafeRaceConditionExample() throws InterruptedException {
    System.out.println("========== UNSAFE RACE CONDITION EXAMPLE ==========");
    UnsafeTicketSalesCounter counter = new UnsafeTicketSalesCounter();

    int numberOfThreads = 4;
    int salesPerThread = 50_000;
    int expectedSales = numberOfThreads * salesPerThread;

    Thread[] sellers = new Thread[numberOfThreads];

    for (int i = 0; i < numberOfThreads; i++) {
        sellers[i] = new Thread(() -> {
            for (int j = 0; j < salesPerThread; j++) {
                counter.sellOneTicketUnsafe();
            }
        }, "UnsafeSeller-" + (i + 1));
        sellers[i].start();
    }

    for (Thread seller : sellers) {
        seller.join();
    }

    System.out.println("Expected sold tickets: " + expectedSales);
    System.out.println("Actual sold tickets:   " + counter.getSoldTickets());
    System.out.println();
}

private static void runSynchronizedRaceConditionExample() throws InterruptedException {
    System.out.println("========== SYNCHRONIZED SOLUTION EXAMPLE ==========");
    SafeTicketSalesCounter counter = new SafeTicketSalesCounter();

    int numberOfThreads = 4;
    int salesPerThread = 50_000;
    int expectedSales = numberOfThreads * salesPerThread;

    Thread[] sellers = new Thread[numberOfThreads];

    for (int i = 0; i < numberOfThreads; i++) {
        sellers[i] = new Thread(() -> {
            for (int j = 0; j < salesPerThread; j++) {
                counter.sellOneTicketSafely();
            }
        }, "SafeSeller-" + (i + 1));
        sellers[i].start();
    }
```

```
        for (Thread seller : sellers) {
            seller.join();
        }

        System.out.println("Expected sold tickets: " + expectedSales);
        System.out.println("Actual sold tickets:   " + counter.getSoldTickets());
    }
}
```

4. Sample Output

The exact output of the concurrent part may change each time because thread scheduling is not fixed. The race condition result may also vary from one execution to another.

4.1 Sequential Processing Output

```
========== SEQUENTIAL TICKET PROCESSING ==========
Starting Operation 1: Book 10 ticket(s)
Finished Operation 1
Available tickets: 90

Starting Operation 2: Cancel 3 ticket(s)
Finished Operation 2
Available tickets: 93

Starting Operation 3: Check available tickets
Ticket check: 93 ticket(s) available.
Finished Operation 3
Available tickets: 93

...

Final available tickets after sequential processing: 70
```

4.2 Concurrent Processing Output

```
========== CONCURRENT TICKET PROCESSING ==========
TicketWorker-4 started Operation 4: Reserve 5 ticket(s) for organiser hold
TicketWorker-1 started Operation 1: Book 10 ticket(s)
TicketWorker-6 started Operation 6: Cancel 2 ticket(s)
TicketWorker-3 started Operation 3: Check available tickets
TicketWorker-7 started Operation 7: Check available tickets

Ticket check: 100 ticket(s) available.
TicketWorker-7 updated inventory. Available tickets: 100
TicketWorker-7 finished Operation 7

...

Final available tickets after concurrent processing: 70
```

4.3 Unsafe Race Condition Output

```
========== UNSAFE RACE CONDITION EXAMPLE ==========
Expected sold tickets: 200000
Actual sold tickets:   50078
```

4.4 Synchronized Solution Output

```
========== SYNCHRONIZED SOLUTION EXAMPLE ==========
Expected sold tickets: 200000
Actual sold tickets:   200000
```

5. Report Explanation

5.1 Introduction

This project develops a Java console-based Concurrent Concert Ticket Booking System. The system simulates ticket operations such as booking tickets, cancelling tickets, checking availability, and reserving tickets for organiser use. The main purpose is to demonstrate how Java threads can process tasks concurrently and how synchronization protects shared data from race conditions.

5.2 Sequential Transaction Processing

In the sequential version, all ticket operations are processed one after another. The next operation starts only after the previous operation has completely finished. This makes the output predictable because there is only one execution flow.

For example, if Operation 1 books 10 tickets, the available ticket count is updated before Operation 2 starts. Therefore, there is no overlapping access to the shared ticket inventory.

5.3 Concurrent Transaction Processing

In the concurrent version, each ticket operation is assigned to a separate thread using the Runnable interface. Each TicketWorker object processes one operation. Calling start() on a thread creates a new thread of execution. This allows several ticket operations to be active at overlapping times.

The output order may change every time the program runs because the operating system and JVM decide when each thread gets CPU time. Therefore, TicketWorker-1 may not always finish before TicketWorker-2.

Calling run() directly would not create a new thread. It would behave like a normal method call and execute in the current thread. Therefore, start() must be used to achieve actual multithreaded execution.

5.4 Shared Data, Stack, and Heap Explanation

Local variables such as operation, threadName, currentValue, and loop counters are private to each thread because they are stored in that thread's stack.

Objects such as TicketInventory, TicketOperation, UnsafeTicketSalesCounter, and SafeTicketSalesCounter are stored in the heap. When multiple threads use the same TicketInventory object, they share the same availableTickets variable.

Local variables are usually safer in multithreaded programs because each thread has its own copy. Shared heap data can create problems because multiple threads may read and update the same value at the same time.

5.5 Race Condition Demonstration

The unsafe version uses the following method:

```
public void sellOneTicketUnsafe() {
    int currentValue = soldTickets;
    Thread.yield();
    soldTickets = currentValue + 1;
}
```

This operation is unsafe because incrementing soldTickets is not a single atomic operation. It involves reading the current value, calculating the new value, and writing the new value back.

If two threads read the same old value before either thread writes the new value, one update may be lost. This is why the actual number of sold tickets may be lower than the expected number.

This problem is called a race condition because the final result depends on the timing and order of thread execution.

5.6 Synchronization Solution

The synchronized version uses the following method:

```
public synchronized void sellOneTicketSafely() {
    int currentValue = soldTickets;
    soldTickets = currentValue + 1;
}
```

The critical section is the part of the code that reads and updates the shared variable soldTickets.

Synchronization ensures that only one thread can enter this method at a time for the same object. Therefore, one thread must finish updating soldTickets before another thread can update it. This prevents lost updates and gives the correct final result.

6. Testing and Reflection

What is the main difference between sequential and concurrent ticket processing?

Sequential processing handles one operation at a time. Concurrent processing allows multiple operations to run at overlapping times using threads.

What was observed when multiple threads performed ticket operations?

The start and finish order of operations was not fixed. Different threads could complete at different times depending on scheduling and sleep duration.

Why is the output order not always fixed in concurrent programs?

The JVM and operating system schedule threads dynamically. A thread that starts earlier may finish later if it sleeps longer or receives CPU time later.

What problem happened in the unsafe ticket sales counter?

The actual number of sold tickets could be lower than the expected value because some increments were lost.

How did synchronization solve the problem?

Synchronization allowed only one thread to update the shared soldTickets variable at a time, preventing lost updates.

What does this project teach about safe concurrent Java applications?

It shows that shared heap data must be protected when multiple threads update it. Threads can improve responsiveness, but synchronization is needed for correctness.

7. README File Sample

```
Project Title:
Concurrent Concert Ticket Booking System

How to Compile:
javac ConcertTicketBookingSystem.java

How to Run:
java ConcertTicketBookingSystem
```

Program Sections:
1. Sequential ticket processing
2. Concurrent ticket processing using threads
3. Unsafe race condition demonstration
4. Synchronized solution

Notes:
The concurrent output order may be different each time the program runs.
The unsafe race condition result may also vary between runs.

8. Group Contribution Table Sample

Student Name	Student ID	Main Responsibilities	Estimated Contribution
Student 1	XXXXX	System design and sequential processing	25%
Student 2	XXXXX	Concurrent processing using threads	25%
Student 3	XXXXX	Race condition and synchronization	25%
Student 4	XXXXX	Report, screenshots, README, testing	25%

Total contribution: 100%.

9. Suggested Submission Folder Structure

```
CSC2044_Assignment_GroupXX/
│
├── ConcertTicketBookingSystem.java
├── README.txt
├── Report.pdf
└── screenshots/
    ├── sequential_output.png
    ├── concurrent_output.png
    ├── unsafe_race_condition_output.png
    └── synchronized_solution_output.png
```

10. Reference Note

This sample document follows the general structure and learning focus of the uploaded CSC2044 group assignment, but it uses a different problem scenario and different program logic. The original uploaded assignment was based on a concurrent bank account transaction system; this sample solution is based on a concurrent concert ticket booking system.